DIPUTACIÓN
DE JAÉN

EJERCICIO MATERIAS ESPECÍFICAS PARA LA PROVISIÓN DE UNA PLAZA DE FUNCIONARIA/O DE CARRERA, DE LA DIPUTACIÓN PROVINCIAL DE JAÉN Y SUS ORGANISMOS AUTÓNOMOS, PERTENECIENTES AL GRUPO A, SUBGRUPO A2, ESCALA ADMINISTRACIÓN ESPECIAL, SUBESCALA SERVICIOS ESPECIALES, CLASE COMETIDOS ESPECIALES, DENOMINACIÓN ANALISTA PROGRAMADORA/O MEDIANTE SISTEMA DE OPOSICIÓN LIBRE.

INSTRUCCIONES

1. El examen se realizará con bolígrafo permanente, en ningún caso con lápiz.

2. Rellene los **datos personales y firma*** de la hoja de identificación, en la cual deberá igualmente identificarse con un **número aleatorio de tres dígitos como máximo**.
    - *La firma, ni las hojas del ejercicio, no deberá tener ningún carácter o marca diferenciadora.
    - El número aleatorio junto con la firma deberá repetirse en todas las hojas del ejercicio que se entregue para su emparejamiento.

3. Dispone de 10 minutos para leer el examen antes del comienzo del ejercicio.

4. El valor de cada pregunta está Indiquedo en el enunciado de esta y desglosado en cada uno de sus apartados, siendo la **puntuación máxima del supuesto elegido 20 puntos**.

5. El tiempo de duración del ejercicio será de **ciento veinte minutos**.

6. Desconecte el teléfono móvil. La utilización de cualquier dispositivo móvil durante el ejercicio será causa de expulsión.

7. Cuando finalice, no se levante, alce la mano y un miembro del Tribunal Calificador le recogerá su ejercicio. Deberá entregar todas las hojas con las respuestas.

DIPUTACIÓN
DE JAÉN

SUPUESTO 1

En su trabajo diario en la **Diputación Provincial de Jaén** va a formar parte del equipo de desarrollo y mantenimiento del **sistema de gestión de personal y nómina**. Un trabajo crítico mensualmente es la elaboración de la nómina y liquidación de los seguros sociales, por tanto, el sistema debe realizar dichas tareas de forma correcta. Por otro lado, se necesita gestionar la jornada laboral de todos/as sus empleados/as a través de un **sistema de control horario**.

BLOQUE 1. NÓMINA Y SEGURIDAD SOCIAL (Total 6,0 puntos)

1. **Respecto a la elaboración de la nómina (3,0 puntos)**

    a. Describa cómo es el proceso de elaboración de nómina (1,0 punto)

    b. ¿Qué son los devengos? (0,5 puntos)

    c. ¿Y los descuentos? (0,5 puntos)

    d. ¿Qué es el importe líquido? (0,5 puntos)

    e. ¿Qué es el pago de atrasos? (0,5 puntos)

2. **Respecto a la liquidación de los seguros sociales (3,0 puntos)**

    a. Describa cómo es el proceso de liquidación de seguros sociales (1,0 punto)

    b. Si se deben pagar atrasos del mes anterior al actual, ¿qué tipo de liquidación de atrasos se debe generar? (0,5 puntos)

    c. ¿Qué es el Sistema de Liquidación Directa (SLD)? (0,5 puntos)

    d. ¿Qué es el fichero de bases? (0,5 puntos)

    e. ¿Qué son los ficheros CRA? (0,5 puntos)

DIPUTACIÓN
DE JAÉN

BLOQUE 2. CONTROL HORARIO (Total 6,0 puntos)

3. Aplicación de control horario (6,0 puntos)

a. Programando en I-TIME, tenemos en la variable 'fichaje' el marcaje realizado por un empleado. Si queremos saber la hora en la que se ha realizado dicho marcaje, ¿qué función debemos utilizar? (0,5 puntos)

b. La siguiente instrucción en I-TIME, ¿qué hace? (0,5 puntos)

```
SI (cantidadMovimientos(16:00,19:00) == 0 Y haVenido())
ENTONCES
creaMovimiento(18:00,0);
FSI;
```

c. La siguiente instrucción en I-TIME, ¿qué hace? (0,5 puntos)

```
SI (getDay()==5) ENTONCES
C[C2]=C[C2]+(C[C1] REB 06:00);
SINO FSI;
```

d. En I-TIME, ¿cómo se denomina un contador cuyo valor será arrastrable con la posibilidad de iniciarlo en un periodo determinado? (0,5 puntos)

e. ¿Qué devuelve la función `entero hayAnomalia(<anomalia>)` de I-TIME? (0,5 puntos)

f. En I-TIME, ¿qué es un contador? (0,5 puntos)

g. En I-TIME, Indique los tipos de contadores existentes. (0,5 puntos)

h. ¿Qué tipo de valores puede tomar un contador? (0,5 puntos)

i. ¿Qué herramienta permite programar acciones para cada una de las cuatro fases principales del cálculo, modificando el comportamiento habitual del control horario y adaptarlo a necesidades concretas de la instalación? (0,5 puntos)

j. ¿Qué significa congelar un periodo? (0,5 puntos)

k. ¿Qué es un convenio? (0,5 puntos)

l. ¿Para qué es útil un convenio? (0,5 puntos)

DIPUTACIÓN
DE JAÉN

BLOQUE 3. BBDD (Total 4,0 puntos)

4. Preguntas de BBDD (4,0 puntos)

En la base de datos actual ORACLE llamada “dbdipjaen” se cuenta con el siguiente modelo de datos:

TABLA TIPO_BOLSA:

codibolsa	descbolsa
1	Albañil
2	Jardinero
3	Fontanero
4	Carpintero

TABLA CONVOCATORIA:

codibolsa	codiconvo	fechdesde	fechhasta
3	1	01/01/2025	31/12/2025
4	1	01/01/2025	NULL
3	2	01/01/2026	NULL
1	1	25/04/2026	31/12/2026

TABLA BOLSA:

codibolsa	codiconvo	idperson	numeorden
3	1	500	2
3	1	501	1
4	1	700	1
4	1	701	2
3	2	500	1
3	2	501	2
3	2	502	3
1	1	100	2
1	1	150	1
1	1	700	3

a. Escriba la consulta en PL/SQL que permite saber cuántas convocatorias tiene cada bolsa (1,0 punto).

b. Escriba la consulta en PL/SQL para obtener el número de integrantes de cada bolsa en la convocatoria vigente a día de hoy. (1,0 punto).

c. Escriba la consulta en PL/SQL para obtener listado de bolsas código y descripción de aquellas bolsas que tienen alguna convocatoria vigente (1,0 punto).

d. Dados los siguientes tipos de datos y la definición de la función get_infoBolsas

```
create or replace type info_bolsas_row as object (
    codibolsa tipo_bolsa.codibolsa%TYPE,
    descbolsa tipo_bolsa.descbolsa%TYPE,
    numconvos integer
);

create or replace type info_bolsas_tab as table of info_bolsas_row;

create or replace function get_infoBolsas(p_fecharef date)
return info_bolsas_tab pipelined is

v_numconvos integer;
begin
  for rec in (select * from tipo_bolsa)
  loop
        select case when count(*) is null then 0 else count(*) end
        into v_numconvos
        from convocatoria
        where  codibolsa = rec.codibolsa and
               fechdesde <= p_fecharef and
               (fechhasta >= p_fecharef or fechhasta is null);

     PIPE ROW (info_bolsas_row(rec.codibolsa,rec.descbolsa,v_numconvos));
  end loop;
end;
```

¿Cuál es el resultado de ejecutar la siguiente consulta? (1,0 punto).

```
select * from table(get_infoBolsas('01/01/2026'))
```

DIPUTACIÓN
DE JAÉN

BLOQUE 4. LEGISLACIÓN (Total 4,0 puntos)

5. Responda a las siguientes cuestiones (4,0 puntos).

a. Con respecto al Punto de Acceso General (1,0 punto)
   i. Explique brevemente qué es y cuál es su finalidad (0,85 puntos)
   ii. Indique en qué normativa se regula (0,15 puntos)

b. Explique brevemente qué es y cuál es la finalidad de los siguientes conceptos (1,0 punto)
   i. PID (0,50 puntos)
   ii. SIR (0,50 puntos)

c. Con respecto al REA (1,0 punto)
   i. Explique brevemente en qué consiste (0,85 puntos)
   ii. Indique en qué normativa se regula (0,15 puntos)

d. Con respecto a PLCASP (1,0 punto)
   i. Explique brevemente en qué consiste (0,50 puntos)
   ii. Indique quién o quiénes son los destinatarios (0,35 puntos)
   iii. Indique en qué normativa se regula (0,15 puntos)

DIPUTACIÓN
DE JAÉN

SUPUESTO 2

La **Diputación Provincial de Jaén** necesita una aplicación web para que los empleados puedan consultar y gestionar sus solicitudes de permisos y vacaciones. Entre otras, podrán solicitar días de vacaciones y asuntos particulares. Cada solicitud contiene el empleado, la fecha solicitada, el tipo de permiso y su estado.

Los estados posibles son PENDIENTE (pendiente de revisión), APROBADA (autorizada) y RECHAZADA (no autorizada). La aplicación se desarrollará con Angular en el frontend y Spring Boot en el backend, que expondrá una API REST.

Para los ejercicios de frontend se utilizará la siguiente interfaz y los datos de ejemplo:

```
export interface SolicitudPermiso {
  id: number;
  empleado: string;
  fechaSolicitud: string;  // Formato yyyy-MM-dd
  tipo: string;            // 'VACACIONES' o 'ASUNTOS_PROPIOS'
  estado: string;          // 'PENDIENTE', 'APROBADA' o 'RECHAZADA'
}

export const SOLICITUDES: SolicitudPermiso[] = [
  { id: 1, empleado: 'Ana García', fechaSolicitud: '2025-10-15',
    tipo: 'VACACIONES', estado: 'APROBADA' },
  { id: 2, empleado: 'Juan López', fechaSolicitud: '2025-11-01',
    tipo: 'ASUNTOS_PROPIOS', estado: 'PENDIENTE' },
  { id: 3, empleado: 'María Torres', fechaSolicitud: '2026-09-20',
    tipo: 'VACACIONES', estado: 'RECHAZADA' },
  { id: 4, empleado: 'Ana García', fechaSolicitud: '2026-12-05',
    tipo: 'ASUNTOS_PROPIOS', estado: 'PENDIENTE' },
  { id: 5, empleado: 'Carlos Ruiz', fechaSolicitud: '2026-10-30',
    tipo: 'VACACIONES', estado: 'APROBADA' }
];
```

BLOQUE 1. FRONTEND: ANGULAR (Total 6,0 puntos)

1. Filtrado y ordenación de solicitudes (2,5 puntos)

a. Implemente una función en TypeScript que reciba un array de solicitudes, un texto de búsqueda y un estado, y devuelva las solicitudes que cumplan ambas condiciones, ordenadas de la más reciente a la más antigua según fechaSolicitud. (1,0 punto)

   Requisitos: El texto se buscará en el nombre del empleado, sin distinguir mayúsculas de minúsculas y permitiendo búsquedas parciales. Si está vacío o contiene solo espacios, no se filtrará por empleado. Si el estado es una cadena vacía, no se filtrará por estado; en otro caso, solo se devolverán las solicitudes con ese estado. No se debe modificar el array original.

```
function filtrarSolicitudes(
  solicitudes: SolicitudPermiso[],
  texto: string,
  estado: string
): SolicitudPermiso[] {
  // Completar
}
```

b. Escriba una instrucción utilizando filter() que obtenga únicamente las solicitudes pendientes y otra utilizando map() que devuelva un array con los nombres de los empleados. (0,5 puntos)

c. En Angular, ¿qué función tienen los decoradores @Input() y @Output() en un componente? Explique brevemente cómo se utilizan para comunicar un componente hijo con su componente padre. (0,5 puntos)

d. Actualice la interfaz SolicitudPermiso para añadir el campo fechaSolicitudFin, que será opcional y permitirá indicar la fecha de finalización de una solicitud de vacaciones. (0,5 puntos)

2. Servicio de comunicación con el backend (1,5 puntos)

Se dispone del siguiente esqueleto de servicio Angular para comunicarse con la API REST de Spring:

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { SolicitudPermiso } from './solicitud.model';

@Injectable({ providedIn: 'root' })
export class SolicitudesService {

  // Endpoints de la API REST
  private readonly LISTADO = '/api/solicitudes/listado';
  private readonly ALTA = '/api/solicitudes/alta';

  constructor(private http: HttpClient) {}
}
```

a. Implemente el método `obtenerSolicitudes(): Observable<SolicitudPermiso[]>` que realice una petición al endpoint LISTADO para recuperar todas las solicitudes. (0,5 puntos)

b. Implemente el método `registrarSolicitud(solicitud: SolicitudPermiso): Observable<SolicitudPermiso>` que realice una petición al endpoint ALTA para dar de alta una nueva solicitud en el backend. (0,5 puntos)

c. ¿Qué es un Observable en Angular y para qué se utiliza habitualmente junto con HttpClient? (0,25 puntos)

3. Componente de listado (2,0 puntos)

El componente debe almacenar en la propiedad solicitudes, las solicitudes recibidas del backend para poder mostrarlas en pantalla.

```
export class SolicitudesComponent implements OnInit {
  solicitudes: SolicitudPermiso[] = [];

  constructor(private servicio: SolicitudesService) {}
  ngOnInit(): void {
    this.servicio.obtenerSolicitudes().subscribe({
      next: (datos) => { console.log(datos); },
      error: (error) => { console.log(error); }
    });
  }
}
```

a. ¿Qué modificación debe realizarse para que las solicitudes se almacenen en la propiedad del componente? (0,5 puntos)

b. ¿Qué ventaja tiene realizar esta consulta en ngOnInit() en lugar de hacerlo directamente en el constructor? (0,5 puntos)

c. Escriba la plantilla HTML del componente (<dpj-solicitudes>) que muestre las solicitudes en una tabla con las columnas empleado, tipo, fecha de solicitud y estado, usando *ngFor (o @for). La fecha se mostrará con el pipe date:'dd/MM/yyyy'. Además, cree y aplique las clases CSS .estado-aprobada (texto verde) y .estado-rechazada (texto rojo) según el estado de cada solicitud. (1,0 punto)

DIPUTACIÓN
DE JAÉN

BLOQUE 2. BACKEND: SPRING BOOT (Total 6 puntos)

Se proporciona un proyecto base con las dependencias necesarias ya configuradas.

4. DTO y validaciones en Spring Boot (1 punto)

Se proporciona la clase SolicitudDTO, que representa los datos de una solicitud de permiso o vacaciones que recibe el backend.

a. Cree la clase SolicitudDTO con los siguientes atributos (0,25 puntos)

```
empleado: nombre del empleado (String), obligatorio y no puede estar en
blanco.
fechaSolicitud: fecha de la solicitud (LocalDate), obligatoria.
fechaSolicitudFin: fecha de finalización (LocalDate), opcional.
tipo: tipo de solicitud (String), obligatorio y no puede estar en blanco.
estado: estado de la solicitud (String), obligatorio y no puede estar en
blanco.
```

Utilice las anotaciones de validación adecuadas para indicar qué campos son obligatorios.

b. En un método de un controlador REST que recibe un objeto SolicitudDTO mediante una petición POST, indique cómo utilizaría la anotación @Valid para validar automáticamente los datos recibidos (0,25 puntos)

c. Explique brevemente para qué sirven las siguientes anotaciones y en qué se diferencian: @NotBlank y @NotNull. (0,25 puntos)

d. ¿Qué ocurre cuando se recibe una solicitud con un campo obligatorio vacío o nulo y se utiliza @Valid? Indique qué tipo de respuesta HTTP debería devolver el backend (0,25 puntos)

5. Desarrollo de un controlador REST (2,5 puntos)

Se dispone del siguiente controlador:

```
@RestController
@RequestMapping("/api/solicitudes")
public class SolicitudController {

    @Autowired
    private SolicitudService servicio;

    // Completar
}
```

a. Implemente un método que responda a una petición **GET** sobre /api/solicitudes y devuelva una lista de objetos SolicitudDTO. El método deberá delegar la obtención de los datos en SolicitudService. (1,0 punto)

b. Creación de una solicitud. Implemente un método que responda a una petición POST sobre /api/solicitudes para crear una nueva solicitud. (1,0 punto)

   El método deberá:
   - Recibir los datos de la solicitud mediante un objeto SolicitudDTO.
   - Utilizar @Valid para validar los datos recibidos.
   - Delegar la creación de la solicitud en SolicitudService.
   - Devolver como respuesta la solicitud creada.

c. Explique la diferencia entre GET y POST e indique un uso de cada uno en esta aplicación. (0,5 puntos)

6. Detección y corrección de errores en el servicio (2,5 puntos)

Se dispone del siguiente repositorio y del método que debe modificar el estado de una solicitud:

```
public interface SolicitudRepository
        extends JpaRepository<Solicitud, Long> {
}

public SolicitudDTO actualizarEstado(Long id, String estado) {
    Solicitud solicitud = repositorio.findById(id);
    solicitud.setEstado(estado);
    return convertirADTO(solicitud);
}
```

a. Identifique los errores existentes en el método anterior e indique qué problemas podrían producirse durante su ejecución. (1,0 punto)

b. Corrija el método para que busque la solicitud, compruebe si existe, actualice su estado, guarde los cambios y devuelva el DTO. (1,0 punto).

c. Explique por qué el Controller no debería acceder directamente al Repository cuando existe una capa Service. (0,5 puntos).

DIPUTACIÓN
DE JAÉN

BLOQUE 3. BBDD (Total 4,0 puntos)

7. Preguntas de BBDD (4,0 puntos)

En la base de datos actual ORACLE llamada “dbdipjaen” se cuenta con el siguiente modelo de datos:

TABLA TIPO_BOLSA:

codibolsa	descbolsa
1	Albañil
2	Jardinero
3	Fontanero
4	Carpintero

TABLA CONVOCATORIA:

codibolsa	codiconvo	fechdesde	fechhasta
3	1	01/01/2025	31/12/2025
4	1	01/01/2025	NULL
3	2	01/01/2026	NULL
1	1	25/04/2026	31/12/2026

TABLA BOLSA:

codibolsa	codiconvo	idperson	numeorden
3	1	500	2
3	1	501	1
4	1	700	1
4	1	701	2
3	2	500	1
3	2	501	2
3	2	502	3
1	1	100	2
1	1	150	1
1	1	700	3

a) Escriba la consulta en PL/SQL que permite saber cuántas convocatorias tiene cada bolsa (1,0 punto).

b) Escriba la consulta en PL/SQL para obtener el número de integrantes de cada bolsa en la convocatoria vigente a día de hoy. (1,0 punto).

c) Escriba la consulta en PL/SQL para obtener listado de bolsas código y descripción de aquellas bolsas que tienen alguna convocatoria vigente (1,0 punto).

d) Dados los siguientes tipos de datos y la definición de la función get_infoBolsas

```
create or replace type info_bolsas_row as object (
    codibolsa tipo_bolsa.codibolsa%TYPE,
    descbolsa tipo_bolsa.descbolsa%TYPE,
    numconvos integer
);

create or replace type info_bolsas_tab as table of info_bolsas_row;

create or replace function get_infoBolsas(p_fecharef date)
return info_bolsas_tab pipelined is

v_numconvos integer;
begin
  for rec in (select * from tipo_bolsa)
  loop
        select case when count(*) is null then 0 else count(*) end
        into v_numconvos
        from convocatoria
        where  codibolsa = rec.codibolsa and
               fechdesde <= p_fecharef and
               (fechhasta >= p_fecharef or fechhasta is null);

     PIPE ROW (info_bolsas_row(rec.codibolsa,rec.descbolsa,v_numconvos));
  end loop;
end;
```

¿Cuál es el resultado de ejecutar la siguiente consulta? (1,0 punto).

```
select * from table(get_infoBolsas('01/01/2026'))
```

DIPUTACIÓN
DE JAÉN

BLOQUE 4. LEGISLACIÓN (Total 4,0 puntos)

8. Responda a las siguientes cuestiones (4,0 puntos).

a. Con respecto al Punto de Acceso General (1,0 punto)
   i. Explique brevemente qué es y cuál es su finalidad (0,85 puntos)
   ii. Indique en qué normativa se regula (0,15 puntos)

b. Explique brevemente qué es y cuál es la finalidad de los siguientes conceptos (1,0 punto)
   i. PID (0,50 puntos)
   ii. SIR (0,50 puntos)

c. Con respecto al REA (1,0 punto)
   i. Explique brevemente en qué consiste (0,85 puntos)
   ii. Indique en qué normativa se regula (0,15 puntos)

d. Con respecto a PLCASP (1,0 punto)
   i. Explique brevemente en qué consiste (0,50 puntos)
   ii. Indique quién o quiénes son los destinatarios (0,35 puntos)
   iii. Indique en qué normativa se regula (0,15 puntos)